

HW1-4 重点回答

1. RadixAttention / RadixCache 解决什么问题？

LLM 服务中大量请求共享前缀（system prompt、few-shot、多轮对话历史）。朴素 KV cache 只能按请求整体复用。RadixCache 用基数树按 token 序列组织 KV，任意两个请求的最长公共 token 前缀都可自动复用，从而减少重复 Prefill 计算、降低 TTFT、提高吞吐。

2. page-sized KV cache 与 prefix reuse 对应 pipeline 哪一部分？

page-sized KV cache 对应 KV Cache 管理层（token pool 的分页分配，RadixCache 的底层存储，页为最小复用/淘汰单位，类似 vLLM PagedAttention 的页表机制）；prefix reuse 对应调度阶段的 RadixCache。match_prefix 与请求结束时的 cache_finished_req 写回，即 Prefill 之前的复用判断与完成后的缓存更新。

3. vLLM PagedAttention 解决什么问题？与 RadixAttention 的联系与差异？

PagedAttention 解决 KV cache 显存碎片与低利用率问题：借鉴操作系统分页，把每请求 KV 切成固定大小页，按需分配、逻辑连续物理离散，使显存浪费从远超 100% 降到 4% 以下。联系：二者都是 KV 显存的分页化管理；SGLang 的 page-sized KV cache 正是借鉴了 PagedAttention。差异：PagedAttention 关注"怎么存"（显存利用率），本身不提供跨请求复用；RadixAttention 在分页之上增加"复用什么"（radix 树索引公共前缀，实现自动前缀共享）。两者互补——现代推理系统通常同时采用。